Can a Transformer Learn Iterative Algorithms? Briefly summarized,[1] a transformer is composed of a series of transformer blocks and operates on the space of sequences. A transformer block performs cross-operations that combine elements of a sequence through the use of *attention heads* to generate new sequences, and parallel operations applied to each element of a sequence separately through the use of *feedforward layers* (or MLP, i.e., multi-layer perceptrons). Auto-regressive transformers in particular are trained to perform next-token prediction; in other words, from a training corpus that contains sequences (z_t) of tokens, the transformer is trained to output z_{t+1} from the truncated sequence $S_t = (z_r)_{r \in [t]}$.

Given a certain number of transformer blocks, a transformer can only apply a corresponding number of cross-operations to predict the next token. This limits its ability to learn even relatively simple iterative tasks, see Figures 2 and 3. E.g., consider the task where the input sequence is $(x_1, \ldots, x_L)$, possibly restricted to $x_i \in [a, b]$ for some $a < b$, and the desired output is the product $\prod x_i$. The product is multilinear in the entries of the input sequence $(x_1, \ldots, x_L)$. If we model the output of an attention layer as sums of monomials of degree at most three in its input variables (due to key-query-value interaction), this makes learning the task quite hard for a transformer, and bounds the maximum length of the sequences that a transformer with a given number of blocks can correctly process [see 51, and related literature for formal discussions on the matter that capture this log-depth dependency].

However, when transformers are allowed to generate many tokens before providing an answer, which implicitly lifts the constraint on the number of operations performed by the transformer (see Figure 4), the picture changes [48, 39, 35] [see also 14, 21]. In particular, Figure 5, explained in the next subsection, illustrates how a two-layer transformer can implement what we named an "iteration head". This potentially enables it to learn any iterative algorithm, assuming that its second layer MLP is big enough to implement any successor function $F : (x_t, s_{t-1}) \mapsto s_t$.

Synthetic Data. To study the emergence of CoT in controlled settings, we introduce two simple iterative problems. The first problem is a straightforward instance of Algorithm 1, where the tokens and the states are elements of the finite field $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ (for some prime number p), i.e., integers modulo p, and the iterative step is the evaluation of a polynomial function $P \in \mathbb{F}_p[X, Y]$ in those two variables:

$$x \in \mathbb{F}_p, \quad \texttt{Init} = 0, \qquad F(s, x) = P(s, x) \qquad \text{(Polynomial Iteration)}$$

Letting $P(s, x) = s + x$ and $p = 2$, the problem reduces to the so-called parity problem:

$$x \in \{0, 1\}, \quad \texttt{Init} = 0, \qquad F(s, x) = s + x. \qquad \text{(Parity Problem)}$$

For ease of study, we also consider an even simpler problem: the copying problem, where the goal is simply to output an exact copy of the input sequence.

$$x \in \{0, 1\}, \qquad F(s, x) = x \qquad \text{(Binary Copy)}$$

Note that there is a small abuse of notation here, since we are interested in the unrolled sequence of states produced iteratively by Algorithm 1, rather than the last token only. While copying may seem like an overly simplistic task, it should be put in parallel with the seminal work of Olsson et al. [44] that advocates studying a copying mechanism to better understand in-context learning.

For each of our problems, we encode the data, i.e. the sequences (z_t), in the following form:

$$[\texttt{Problem}] \quad [x_1] \quad [x_2] \quad \cdots \quad [x_L] \quad [\texttt{EoI}] \quad [s_1] \quad [s_2] \quad \cdots \quad [s_L] \quad [\texttt{EoS}].$$

A first token indicates the problem generating the sequence (e.g., "copy", or "parity"), after which L input tokens x_t are provided. The end of the input is specified by an end-of-input token (EoI). Subsequent tokens encode the states s_t of Algorithm 1 at each iteration, until termination, which is indicated by an end-of-sequence token (EoS).

3 One Head to Rule Them All

We have discussed how transformers are limited in the iterative tasks that they can efficiently solve using only next-token prediction. By contrast, we describe in this section a certain distribution of weights which, if correctly learnt, would allow a two-layer transformer to efficiently implement iterative algorithms by using chain-of-thought reasoning. After that, we perform various experiments to identify the conditions under which this theoretical circuit does appear.

[1] We assume that the reader is familiar with the transformer architecture [see, e.g., 56, 36, for details].

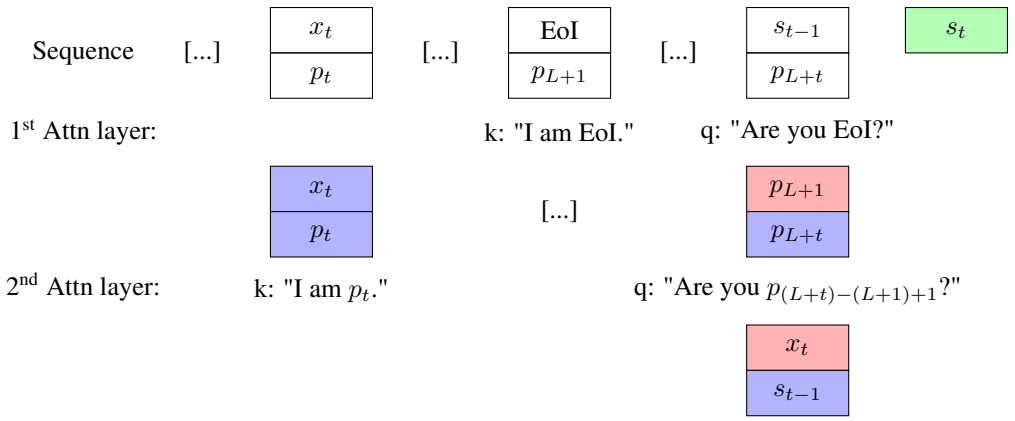

Figure 5: Implementation of an iteration head with a two-layer transformer. Contiguous box: superposition in high-dimensional space. Blue: information brought to working space thanks to residual connections. Red: information brought thanks to attention. Green: next-token prediction. The first layer MLP implements a subtraction $t = (L + t) - (L + 1) + 1$ for the second attention to be able to query p_t from (p_{L+1}, p_{L+t}). The second layer MLP implements F to be able to predict s_t from (s_{t-1}, x_t), with the "end-of-input" mark assimilated to the initial state s_0 of Algorithm 1.

3.1 Theoretical Circuit

This subsection describes a natural way to implement an iterative algorithm with a transformer. Let us consider a prompt $(x_t)_{t\in[L]}$, to which we append a special "end-of-input" (EoI) token that marks the end of the input. The completion sequence will be generated with the t-th new element encoding for the state variable after t steps of Algorithm 1. The t-th new element (i.e. the $L + 1 + t$-th token of the full sequence) is produced as follows. The first attention head is tasked with retrieving the position of the end of the initial prompt, i.e. the position of the EoI token. As illustrated in Figure 5, it does so using a query-key combination which informally encodes the question "Are you EoI?" and the answer "I am EoI.". Thus it extracts the positional encoding p_{L+1} (which is the value associated to the $L + 1$-th token) regardless of the sequence length $L \in \mathbb{N}$, and brings p_{L+t} into its working space as well through the residual connection (we formalize such statements in the next paragraph). As shown further below in Figure 5, the next attention head then generates a query "Are you p_t?" from p_{L+1} and p_{L+t}, which is answered positively by a key "I am p_t" associated to the t-th position. Hence the head retrieves the value associated to this position, which is x_t. It also obtains s_{t-1} (or rather the approximation of it that was produced at the previous step) through the residual stream. The MLP can finally compute the new state $s_t = F(s_{t-1}, x_t)$ from s_{t-1} and x_t. This can always be done by a large-enough MLP assuming that the second attention layer outputs all the relevant information regarding s_{t-1} and x_t, as a result of universal approximation [27]. Note that the operations performed by the two attention layers are totally independent from the precise iterative task considered, i.e. from the choice of F; their only goal is to retrieve x_t and s_{t-1}. We call the pattern of weights that realize these operations, as well as the underlying algorithm, an "iteration head".

Information Superposition in Working Spaces. In our description of an iteration head, we have rather informally said that some variable x is "extracted" or "obtained". Formally, a transformer transforms a sequence $(x_t)_{t\in[L]}$ into a series of sequences $(e_{t,l})_{t\in[L]}$, where l is an index specifying layers, and $e_{t,l} \in \mathbb{R}^d$, with $\mathbb{R}^d$ being referred to as the "working space". The input tokens and their positions are brought into working spaces using embeddings that are typically learned, then added together. Assuming that the working spaces are high-dimensional enough, and because those embeddings are learned, a transformer can use different parts of $\mathbb{R}^d$ to simultaneously store token and positional information, as if those embeddings were actually *concatenated* rather than added. Likewise, transformer layers output variables are learned functions of their input; if needed, and assuming that d is large enough, $e_{t,l+1}$ can superpose some $e_{s,l}$ and $e_{r,l}$ for different $s, r \leq t$, in which case one may consider $e_{t,l+1}$ as somewhat equivalent to the concatenation of $e_{s,l}$ and $e_{r,l}$. This is why our exposition above focuses on "information pathways", i.e. which variable is generated using which variable, and sentences such as "x_t and s_{t-1} are brought to the working space" should be understood as "some vector encoding the relevant information of both x_t and s_{t-1} is produced".

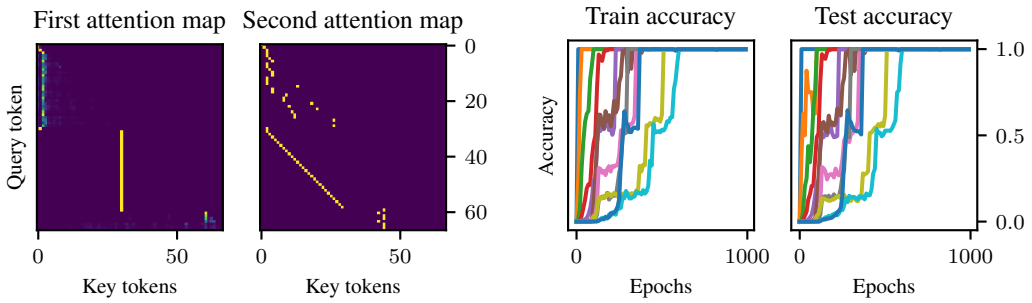

Figure 6: Left: attention maps learned for the parity problem when processing a sequence of length $L = 29$. Yellow indicates high attention score. The yellow line on the left plot shows that all the queries after the EoI token at position $t = 30$ point to the EoI token. In other terms, the first attention implements the "Are you EoI?" query of Figure 5, while the second implements the "Are you p_t?" query. Right: accuracy dynamics for different sequence lengths when learning the parity problem. We observe fast learning of short sequences (we used the `tab10` color scheme of Matplotlib [28] with $L \in \{8, 11, 14, 17, \ldots, 32\}$), and characteristic staircase behaviors.

Approximate Iteration Heads. Iteration heads are an efficient, flexible and parsimonious way to implement iterative algorithms; as such, we expect them to naturally emerge during training. Nonetheless, transformers have flexible architectures that can perform similar operations in different ways. Hence we also expect to see some variations with respect to the schematic architecture described above (see Figures 8 and 13), in particular when the embedding dimension becomes too small for the information superposition from the previous paragraph to be correctly implemented.

3.2 Learning an Iteration Head

In this subsection, we examine the circuit that a transformer actually learns when trained on an iterative task with chain-of-thought. We observe that the theoretical circuit described in the previous subsection does appear in practice.

Experimental Design. Unless otherwise stated, our experimental setup is as follows. Data was generated for the binary-copy, parity, and polynomial iteration problem with $P(X, Y) = XY + 1$ in $\mathbb{F}_{11}$. For each length L from $L_{\min} = 1$ to $L_{\max} = 32$, we generated $n = 1024$ input sequences of length L (corresponding to a total sequence length of $2L + 3$) uniformly at random for both training and testing sets, creating datasets of $N = 16,384 = 16 \times 1024$ sequences in total.[2] We utilized auto-regressive transformers [10] with two layers and one attention head per layer. The embedding dimension was set to $d = 128$, with learned absolute positional encoding added to the learned token embedding. The weights were optimized over 1000 epochs with Adam [29], a batch size of 256, and a fixed learning rate set to $\gamma = 3 \cdot 10^{-4}$, with default PyTorch parameters otherwise [46]. Our source code is available at `https://github.com/facebookresearch/pal`. Our experiments consumed 12k V100-hours.

Attention Heads. In our initial experiment, we trained a transformer to solve either the parity task or the copying task only. The "iteration head" pattern of weights, described in the previous sub-section, can be seen in the attention maps of the first and second attention layers: an example is reported in Figure 6. Namely, we observe that when the model produces the $(L + t + 1)$-th token (meant to be s_t), the following happens. The attention of the first transformer block is fully focused on the position of the EoI token, corresponding to the informal query "Is this token equal to EoI?", creating a yellow line on the left of Figure 6. This allows the first attention layer to retrieve the positional encoding p_{L+1} of the EoI token, in addition to the positional encoding p_{L+t} of the last token of the current sequence (the state s_{t-1}) coming from the residual stream. Using this information, the second attention layer is able to generate the informal query "Is this token in position t?", to extract some encoding of the token x_t. Consequently, the attention of the second layer is fully focused on the position of the t-th entry, creating the yellow off-diagonal line on the second plot of

[2] Note that, even when $x_t \in \mathbb{F}_2$, the expected percentage of collision between the training and testing set decreases exponentially fast with L, ensuring minimal contamination between training and testing.